ChiliEdgeNet: A Lightweight CNN-Transformer Hybrid for On-Device Chili Pepper Disease Recognition

Anonymous Author(s)

Affiliation withheld for double-blind review

Abstract. Early plant-disease diagnosis is crucial for crop protection, yet high-accuracy deep models are usually too heavy for the edge devices farmers actually use. Building on the lightweight RTR_Lite_MobileNet, this paper proposes **ChiliEdgeNet**, a compact CNN-Transformer hybrid: the MobileNetV2 backbone is truncated at `features[1:14]`, yielding a 96-channel 14x14 feature grid (196 tokens) that feeds a small Transformer block integrating three parameter-efficient techniques borrowed from LLM architectures - 2D rotary position embedding (2D-RoPE), grouped-query attention (GQA), and RMSNorm. With **671,432 parameters** (2.7 MB, **35% fewer** than the base architecture), it runs fully offline on a CPU at **20.59 ms/image** and reaches **98.93%** (augmented scenario) on **ChiliEdge-DS**, a chili pepper disease dataset we built and publicly release with this paper (5 classes, 2,461 images field-collected in southern Vietnam, fixed train/val/test split). The headline result is that ChiliEdgeNet is **more accurate and lighter at the same time, with no trade-off**: it reaches 98.93% against 98.12% for the base architecture when reproduced at the same seed 42 under the same protocol (+0.81 percentage points, 3 images out of 373), while using only 671,432 against 1,032,568 parameters - 35% fewer. The methodological contribution is to ask *which* design choices actually pay for themselves, via **retrain-from-scratch ablation** and **per-image paired McNemar testing** on clean images plus two test-time degradations. The answer is mixed and we report it as such: **two of the three LLM techniques are free** - 2D-RoPE matches a learned positional table and GQA matches multi-head attention on every axis, together saving **41,472 parameters** - but **RMSNorm is not**, losing 2.95 pp to LayerNorm under blur ($p = 0.019$) for a saving of only 384 parameters. We report two further negative results: the Transformer block contributes measurably only under low light (+1.61 pp), not on clean images; and the Triplet Attention block inherited from the base architecture harms accuracy under blur and low light and was removed from the design. Two XAI techniques from maximally distinct mechanism families confirm the model attends to genuine disease regions, and it beats a general-purpose vision LLM 98.93% to 78.55% at ~103x lower system-level latency.

Keywords: plant disease diagnosis, lightweight model, edge deployment, CNN-Transformer hybrid, explainable AI.

1. Introduction

Chili pepper diseases substantially reduce crop yield, and farmers need a diagnostic tool that runs directly on an edge device: simultaneously lightweight and accurate, two goals that usually conflict.

Lightweight convolutional networks address the size constraint but, being purely local, struggle with relationships between distant leaf regions; vision transformers capture such global relations but are typically heavier. Hybrid CNN-Transformer designs compromise, yet most prior work reports single-seed accuracy without variance-controlled ablation, multi-technique interpretability, or a comparison against a multimodal foundation model. More fundamentally, such work reports the *assembled* model without measuring whether each component earns its parameters.

This work starts from RTR_Lite_MobileNet [5], which augments MobileNetV2 [16] with attention blocks, and proposes **ChiliEdgeNet**, a sub-700-thousand-parameter hybrid integrating three parameter-efficient LLM techniques. The contributions are:

- **ChiliEdgeNet**: fully offline on CPU at 20.59 ms/image (2.7 MB), 98.93% augmented and 98.39% baseline with **671,432 parameters** - higher than the base architecture [5] (98.12%) while using **35% fewer** parameters - leading the pool of a 27-configuration SOTA reproduction.
- **ChiliEdge-DS**, a field-collected chili disease dataset from southern Vietnam (5 classes, 2,461 images, fixed split; Section 3.1), released on Kaggle [23].
- A **component-level audit**: a test-time degradation benchmark plus **retrain-from-scratch ablation** at *block* level (Tiny Transformer) and *individual technique* level (2D-RoPE, GQA, RMSNorm), every comparison tested per-image on the same 373 images. This yields the central claim: **2D-RoPE and GQA are free - 41,472 parameters saved at no measurable accuracy cost - RMSNorm is not.**
- Two XAI techniques from maximally distinct mechanism families (Occlusion Sensitivity, black-box perturbation; Attention Rollout, reading the Transformer's internals) plus a zero-shot Gemini comparison validate the model.

2. Related Work

Selection criteria for the baseline pool. The 27 configurations in Table 3 come from 15 studies selected by: (i) peer-reviewed work published 2023-2026 on image-based plant-disease classification; (ii) an explicit architectural focus on deployability (lightweight CNN / ViT / hybrid) or a broad CNN-vs-ViT comparison; (iii) enough architectural detail (layer configuration, embedding widths, head counts) to reimplement faithfully without contacting the authors; (iv) coverage of all four directions below rather than only the strongest family. Studies were excluded when they targeted detection/segmentation rather than classification, or when the architecture could not be reconstructed unambiguously. The pool is therefore representative of the *deployment-oriented* literature of the last three years, not an exhaustive survey; each model is reproduced by us

under a common protocol, so Table 3 measures competitiveness under shared conditions, not the authors' published numbers.

Lightweight CNNs keep parameters low with efficient attention or compact blocks. RTR_Lite_MobileNetV2 [5] - our base architecture - augments MobileNetV2 [16] with triplet attention [17] and a spatial-attention residual block. ULEN [8] pushes ultra-lightweight design to 28k parameters, MRW-CNN [4] and Lite-MDC [7] target real-time field deployment, and CustomMNetV3L [12] adapts MobileNetV3-Large. All remain purely convolutional and reason locally. **Vision transformers** capture global relations but are heavier: a plain ViT [15], TrIncNet [3] (Inception-style tokenization), PMVT [6] (mobile-oriented), PDLC-ViT [10] (multitask localization plus classification). **Hybrid CNN-Transformers** are closest to ours: GrapeLeafNet [13] fuses Inception-ResNet with a Shuffle-Transformer, MaizeFormerX [2] adds cross-scale attention, LDI-NET [9] combines CNN and transformer branches for multi-label severity - typically 3-9M parameters. **Broad comparative studies**: Mishra et al. [11] benchmark CNN/AlexNet/ResNet/EfficientNet families, Sujatha et al. [14] pair deep features with SVM classifiers, Maurya et al. [1] build a lightweight meta-ensemble for IoT.

We push the hybrid direction further - under 700 thousand parameters via a well-placed backbone truncation and three parameter-efficient LLM techniques - but differ mainly in **how we evaluate**. Most work above ranks models by clean accuracy on a single split; among the leading models in Table 3, this axis has already maxed out (Section 4.2), so it no longer separates competitive designs or reflects deployment conditions. We replace it with a **test-time degradation benchmark**, **per-image paired McNemar tests**, and a **retrain-from-scratch ablation at both block and technique granularity**.

3. Proposed Method

3.1. Dataset

ChiliEdge-DS comprises leaf and fruit images of five classes - `anthracnose`, `healthy`, `leaf_curl`, `leaf_spot`, `yellow_leaf` - field-collected in southern Vietnam, pre-split into `train`/`val`/`test` (1,597 / 491 / 373; 2,461 total) and preserved throughout so results reflect generalization to unseen data. Images were photographed against a white background to make the diseased region stand out, at ~12,000×9,000 native resolution later resized to 300×300; the dataset is released on Kaggle [23]. The split is close to balanced (largest/smallest class = 1.29 < 1.5), so no class weighting was applied and macro-F1 is not distorted by a dominant class. The white background is a **controlled** acquisition setting unlike a phone camera in the field; Section 4.3 probes part of this gap and Section 6 states what it does not cover.

Table 1. Per-class distribution of ChiliEdge-DS across the fixed split.

Class	Train	Val	Test	Total
anthracnose	264	81	62	407

healthy	334	102	78	514
leaf_curl	328	104	82	514
leaf_spot	326	101	76	503
yellow_leaf	345	103	75	523
Total	**1,597**	**491**	**373**	**2,461**

3.2. Architecture: ChiliEdgeNet

Convolution's limited receptive field keeps MobileNetV2 local, whereas symptoms like leaf curl and yellowing span the leaf and need relations between distant regions. The backbone is therefore truncated and its tail replaced by a Transformer block operating on feature-grid cells as image tokens.

Backbone cut point. We truncate at `features[1:14]`, yielding a **96-channel 14x14** map from a 224x224 input, i.e. **196 tokens**. The choice is mechanistic: in MobileNetV2 `features[14]` is the stride-32 stage, so cutting *after* it collapses the grid to **7x7 (49 tokens)**, at which resolution the convolutional stack has already integrated information across nearly the whole image, leaving the Transformer almost no spatial relation to learn. Every cut from `features[1:8]` to `features[1:14]` preserves the 14x14 grid; a preliminary sweep selected cut 14, the deepest of them. The grid also pays off for interpretability: the Attention Rollout map (Section 5.1) is **14x14 rather than 7x7** - four times finer.

Feature path. A residual block with a spatial attention gate (RES_Block, inherited from [5]) filters features after the stem convolution; a 1x1 convolution compresses 96 channels to the 128-dimensional embedding; the 196 grid cells enter the Transformer block as image tokens; a token-wise mean and a linear layer produce five class scores.

A block we removed. The base architecture [5] also places a Triplet Attention gate [17] after the stem. It is **not** part of ChiliEdgeNet: a preliminary ablation on an otherwise identical model, replicated across two independent runs, showed that removing it makes the network *better* under both blur and low light - a negative result we acted on rather than reported around.

Despite the name, ChiliEdgeNet **is purely a vision model and contains no language component** - no text encoder, no tokenizer, no pretrained language weights. It is trained end-to-end on images only with a cross-entropy loss, like an ordinary classifier. Architecturally the block is a standard Vision Transformer except for **three parameter-efficient techniques borrowed from LLMs**:

- **2D-RoPE** rotates query/key vectors by a coordinate-dependent angle per grid axis, encoding 2D position **without a single learned parameter** [18] - against a learned positional embedding needing a 196x128 = **25,088-parameter** table.
- **GQA** lets four query heads share two key/value heads, saving **16,384** attention parameters [19].
- **RMSNorm** drops LayerNorm's mean-subtraction and shift terms, saving **384** parameters [20].

Together these save **41,856 parameters (5.9% of the model)**; Section 4.4.2 measures, one technique at a time, whether that saving costs accuracy. At embedding dimension 128, one Transformer layer, MLP ratio 2 and dropout 0.1, the model has **671,432 parameters** - **35% fewer** than [5] (1,032,568).

3.3. Evaluation Metrics

We report top-1 accuracy, macro-averaged F1 (so minority classes weigh equally), and total parameter count. Every accuracy figure carries a 95% Wilson confidence interval at n = 373. For reference,

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}} \quad (1)$$

4. Experiments and Results

4.1. Setup

Experiments run on a Kaggle Notebook (NVIDIA Tesla T4) in PyTorch 2.10, training each model under two scenarios - **baseline** and **augmented** - per the configuration in Table 2; validation/test are never augmented. The best-validation checkpoint is reloaded before test evaluation. Every variant - proposed model and all ablations - uses **seed 42** and is re-initialised from it before each run, so variants differ only in the component under study.

Table 2. Training configuration.

Item	Value
Input size	224x224
Baseline transform	Resize, ToTensor, Normalize
Augmented transform	30-degree rotation, ColorJitter, horizontal flip, affine
Epochs	30
Batch size	32
Loss	Cross-entropy, label smoothing 0.05
LR schedule	10% linear warm-up, then cosine decay
Learning rate (backbone / new blocks)	1e-4 / 1e-3
Seed	42 (every variant)

Run count and testing. Results for the proposed model and its ablations come from a **single run (seed 42)**, a deliberate choice compensated by a stricter test protocol. With 373 test images each image is worth 0.27 pp, so a mean ± SD over a few seeds still cannot separate models in the 98-99% band (Section 6). Instead every model-to-model comparison uses a **per-image paired McNemar test** on **exactly the same 373 images**,

which answers *"is this difference within random noise?"* directly without estimating run-to-run variance. Each McNemar entry reports **b / c**: images the first model gets right and the second wrong, and vice versa. Every model in this paper, including the base architecture [5] it is compared against, is evaluated at the same single seed 42 under an identical protocol, so the comparison in Section 4.2 is like-for-like.

Inference latency. The CPU figure used throughout (**20.59 ms/image**, 48.6 images/s) is measured on the CPU of the same Kaggle VM used for training - **Intel(R) Xeon(R) @ 2.00 GHz, 4 logical cores** - with PyTorch 2.10 (FP32), batch 1, 10 warm-up then 50 timed iterations. On that machine a Tesla T4 gives **5.65 ms/image** at batch 1 and **1.03 ms/image** at batch 32. Two caveats: this is a **server-class x86 CPU, not edge hardware** (Section 6), and parameter count is **not** a proxy for speed - holding the grid at 14x14 rather than 7x7 makes the final convolutional stages work over four times as many spatial positions. Parameters govern memory footprint; feature-map resolution governs runtime. We report both.

4.2. Comparison with SOTA Models

The proposed model reaches **98.39%** (macro-F1 0.9846) baseline and **98.93%** (macro-F1 0.9896) augmented. Table 3 places it against all 27 configurations reproduced from 15 SOTA studies under the same protocol: it sits among the accuracy leaders while being among the lightest. Because the baselines are single-run reproductions not individually tuned for this dataset, Table 3 reads as competitiveness under a common protocol, not a definitive ranking of the architectures.

Against the base architecture. This is the comparison the paper's claim rests on. Reproduced at the same seed 42 and under the same protocol, ChiliEdgeNet reaches **98.93%** augmented, higher than RTR_Lite_MobileNet [5] (**98.12%**, macro-F1 0.9821) by **+0.81 pp - 3 images out of 373** - while using only **671,432 against 1,032,568 parameters, 35% fewer**. This is the headline result: more accurate and lighter at the same time, with no trade-off.

The 3-image gap is small and deserves to be read at its true confidence: the two 95% Wilson intervals ([96.18, 99.09] and [97.28, 99.58]) overlap substantially - the ceiling of a 373-image test set cannot resolve two models both near 99% - and RTR_Lite was reproduced only once for comparison (Section 6). The contribution that does not depend on this noise, and is therefore the one the paper emphasizes most, remains **parameter efficiency - the parameter count is an exact figure that needs no significance test.** Clean accuracy is also not the only axis that matters for a field tool; Section 4.3 evaluates degraded conditions, where the two *scenarios* separate sharply even though the *architectures* do not.

Table 3. Two-scenario accuracy, macro-F1, and parameters on the chili dataset. *(Baseline rows are single seed-42 runs under the same protocol; only the most accurate variant of each model family is kept. Differences below ~1 pp correspond to fewer than 4 images out of 373, fall inside the Wilson interval, and should not be read as a reliable ranking - see Section 6.)*

Model	Acc Baseline (%)	Acc Augmented (%)	Macro-F1 (Aug)	Parameters
ChiliEdgeNet (ours)	**98.39**	**98.93**	**0.9896**	**671,432**
RTR_Lite_MobileNet [5]	97.32	98.12	0.9821	1,032,568
GrapeLeafNet [13]	91.15	97.59	0.9769	5,308,035
PMVT-XS [6]	82.31	96.25	0.9644	2,367,515
ResNet-50 [11]	95.98	95.71	0.9586	23,597,957
EfficientNetB0 [11]	94.37	94.37	0.9463	4,055,976
TrIncNet [3]	80.70	90.62	0.9092	7,186,661
InceptionV3 + SVM [14]	93.83	90.35	0.9069	25,112,264
Meta-Ensemble [1]	88.74	88.74	0.8908	1,252,638
CustomMNetV3L [12]	87.67	86.86	0.8639	3,275,909
ULEN [8]	93.03	85.52	0.8603	28,069
MaizeFormerX [2]	79.89	82.04	0.8185	3,553,061
AlexNet [11]	85.52	77.75	0.7747	46,767,493
Lite-MDC [7]	68.10	76.68	0.7658	2,288,965
ViT [15]	72.39	73.99	0.7472	50,013,061
MRW-CNN [4]	74.26	73.73	0.7377	797,413
PDLC-ViT [10]	74.26	70.78	0.7054	17,547,269
LDI-NET [9]	70.78	68.63	0.6839	8,798,597

Error analysis. On the 373 augmented test images the model misclassifies only 4 (Fig. 1). All four involve `yellow_leaf`: two `yellow_leaf` images read as `leaf_curl`, and one `healthy` and one `leaf_spot` absorbed into `yellow_leaf` - the ambiguous chlorosis/curling boundary where a human eye also hesitates. With so few errors the test set has limited resolving power: 98.93% carries a 95% Wilson interval of [97.28%, 99.58%], overlapping every model within ~1 pp in Table 3 (Section 6).

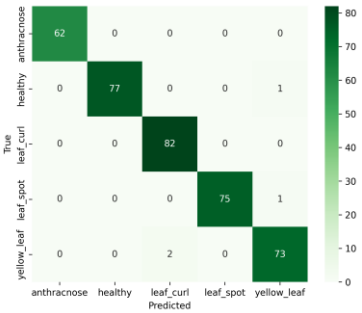


Fig. 1. Confusion matrix of the proposed model on the test set (augmented scenario).

4.3. Robustness to Degraded Capture Conditions

The decisive question for a field tool, as Section 4.2 already posed it, is which model retains accuracy once the image is no longer perfect - not who leads on a clean-image axis that has already saturated. We add degradations applied at **test time** in the spirit of ImageNet-C: neither transform appears during training, and the corrupted sets are generated **once with fixed seeds**, so every model in Sections 4.3-4.4 is scored on pixel-identical inputs.

Table 4. The two degradation conditions and what they model. *(Severity measured over all 373 images.)*

Condition	Real-world factor	Parameter	PSNR	Pixels changed
Gaussian blur	camera shake, defocus	kernel k = 5	33.4 dB	21.41%
Brightness reduction	shooting under foliage or at dusk	factor x0.4	9.9 dB	99.97%

Table 5. The proposed model across the three conditions, with 95% Wilson intervals (n = 373).

Condition	Baseline (%)	Augmented (%)	Wilson 95% (Augmented)
Clean	98.39	**98.93**	[97.28 - 99.58]
Blur (k=5)	90.88	91.42	[88.14 - 93.86]
Low light (x0.4)	89.81	**99.20**	[97.66 - 99.73]
Mean	93.03	**96.51**	-

Table 5 shows what augmentation actually buys, entirely hidden if one looks only at clean images. The two scenarios are within 2 images of each other on clean data and under blur. Under **low light** they differ by **9.38 pp** - 89.81% against 99.20%, i.e. **35 images** - and their Wilson intervals ([86.33, 92.49] against [97.66, 99.73]) do not come close to overlapping. The ColorJitter component teaches tolerance to intensity variation, a real capture condition when photographing under foliage or at dusk. **The proposed model is therefore the augmented one**; all ablations use it.

This cuts against the paper's own architectural framing and is worth stating plainly: on this dataset **low-light robustness is bought by the training recipe, not the architecture**. Section 4.4 measures what the architecture adds on top - and the answer is modest.

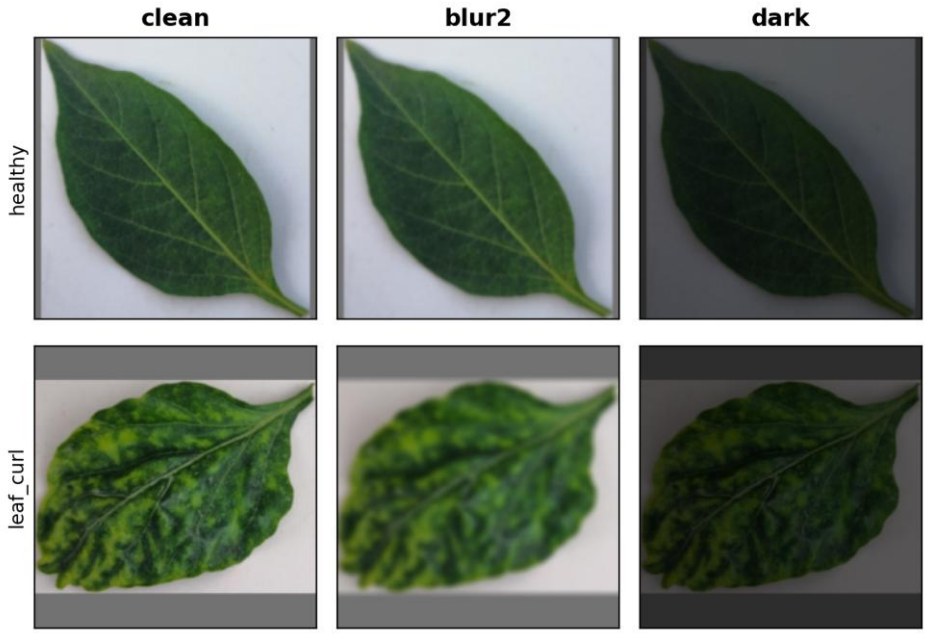


Fig. 2. The test conditions: clean, Gaussian blur (k = 5), and brightness reduction (x0.4), one sample per class.

4.4. Ablation Study

Every variant is **retrained from scratch** with the component under study removed or replaced, using the same seed 42, recipe and 30 epochs; the baseline row is the proposed model trained in the same pipeline. We deliberately do **not** use inference-time knock-out: that measures how much a trained network *depends* on a block, not how much the block *contributes*, and conflating the two is a known methodological error.

4.4.1. Block-Level Ablation

Table 6. Removing each block and retraining (augmented scenario, seed 42).

Variant	Parameters	Clean	Blur	Low light	Mean
Full (proposed)	671,432	**98.93**	**91.42**	**99.20**	**96.51**
w/o Tiny Trans-former	556,104	98.66	90.62	97.59	95.62

Table 7. Per-image paired McNemar tests, full model against each variant. *(Positive delta = the full model is better, i.e. the block contributes. b / c = images the full model gets right and the variant wrong, and vice versa.)*

Block removed	Condition	Delta (pp)	b / c	*p*
Tiny Transformer	**Low light**	**+1.61**	7 / 1	**0.070**
Tiny Transformer	Blur	+0.80	15 / 12	0.701
Tiny Transformer	Clean	+0.27	2 / 1	1.000

The Tiny Transformer's only measurable contribution is under low light, and it is marginal. Removing it costs **1.61 pp** under low light - 7 images broken against 1 improved, $p = 0.070$ - while on clean images it costs **0.27 pp**, a single image, $p = 1.000$, entirely indistinguishable. The direction is consistent (the full model leads on all three axes, and the block is worth 115,328 parameters), but at $p = 0.070$ this is a **trend, not an established effect**. The wider point survives regardless: whatever the Transformer contributes, it is **not** clean accuracy. Correspondingly, the only variant in Table 6 differs from the full model by 0.27 pp - one image - on clean data.

4.4.2. Technique-Level Ablation

The technical contribution named in the title is the integration of **three** parameter-efficient LLM techniques. Block-level ablation cannot separate them: with a single Transformer layer, removing "the Transformer block" removes all three at once. Each variant here replaces **exactly one** technique with its standard counterpart, holds everything else fixed, and retrains from scratch. This section answers whether each technique earns its parameter saving.

Table 8. Replacing each LLM technique with its standard counterpart (augmented scenario, seed 42).

Variant	Replaces	Δ params	Clean	Blur	Low light	Mean
Proposed	2D-RoPE + GQA + RMSNorm	-	98.93	91.42	**99.20**	96.51
`learned_pe`	2D-RoPE swapped for learned PE	**+25,088**	99.20	90.35	98.12	95.89
`mha`	GQA swapped for Multi-Head Attention	**+16,384**	98.93	89.28	98.39	95.53
`layernorm`	RMSNorm swapped for LayerNorm	+384	**99.46**	**94.37**	98.93	**97.59**

Table 9. Per-image paired McNemar tests, proposed block against each single-technique replacement. *(Positive delta = the proposed technique is better.)*

Technique replaced	Condition	Delta (pp)	b / c	*p*
2D-RoPE swapped for learned PE	Low light	+1.07	4 / 0	0.125
2D-RoPE swapped for learned PE	Blur	+1.07	13 / 9	0.523
2D-RoPE swapped for learned PE	Clean	−0.27	0 / 1	1.000
GQA swapped for MHA	Blur	+2.14	16 / 8	0.152
GQA swapped for MHA	Low light	+0.80	3 / 0	0.250
GQA swapped for MHA	Clean	0.00	1 / 1	1.000
RMSNorm swapped for LayerNorm	**Blur**	**−2.95**	4 / 15	**0.019**
RMSNorm swapped for LayerNorm	Clean	−0.54	0 / 2	0.500
RMSNorm swapped for LayerNorm	Low light	+0.27	1 / 0	1.000

The three techniques do not behave alike: **two are free and one is not.**

2D-RoPE is free. It encodes position with **zero learned parameters**; the learned alternative needs a 196x128 = **25,088-parameter** table, 3.7% of the model. No axis separates them (*p* between 0.125 and 1.000); the proposed block leads on both degraded axes and trails by exactly one image on clean. 2D-RoPE therefore buys a 25,088-parameter saving at no measurable accuracy cost - and unlike a learned table, carries no dependence on the token count fixed at training time.

GQA is free. Sharing two key/value heads across four query heads saves **16,384 parameters**, and the proposed block **matches or beats** MHA on all three axes (clean tie; +2.14 pp blur; +0.80 pp low light), none of it significant. The memory advantage that motivated GQA for large LLMs is not material at this scale, but the parameter saving is real and costs nothing here.

RMSNorm is not free. This is the one technique that loses. LayerNorm is **2.95 pp better under blur** ($p = 0.019$, 15 images improved against 4 broken) - the only comparison in this section reaching $p < 0.05$ - and 0.54 pp better on clean, for just **384 extra parameters**. Dropping LayerNorm's mean-subtraction and shift terms saves 0.06% of the model and measurably hurts blur robustness. RMSNorm is thus the weakest of the three choices, and a revision should reconsider it: `layernorm` is in fact the strongest configuration in Table 8 (mean 97.59 against 96.51). We report this against our own architectural preference.

Statistical scope. All results here are **single-seed**. Table 9 comprises 9 tests (3 variants x 3 conditions), so the Bonferroni threshold is $\alpha = 0.05/9 \approx 0.006$; the RMSNorm/blur result ($p = 0.019$) is the only comparison below 0.05 and does **not** survive it. We therefore present this section as **observations that inform design**, not es-

tablished effects. What depends on no test is the arithmetic: 2D-RoPE and GQA together remove **41,472 parameters** and no measurement in Tables 8-9 shows an accuracy cost for either.

5. Model Interpretability

5.1. Explainable AI

High accuracy alone is not enough to trust a model, which could exploit spurious cues such as background - a real risk here, where every image is shot against a white backdrop. We ran eight interpretability methods on the trained model (Grad-CAM [21] and Grad-CAM++, Score-CAM, Integrated Gradients, SHAP, LIME, Occlusion Sensitivity, Attention Rollout) and present the **two whose mechanisms are furthest apart**, since they are unlikely to fail in the same way. **Occlusion Sensitivity** [24] is a **perturbation** method: it slides a grey patch over the image and measures the drop in true-class probability, **never looking inside the model** - no gradients, no weights, no activations - so it is evidence independent of any architectural assumption. **Attention Rollout** [22] does the opposite: it reads the attention matrices **inside** the Transformer block and traces attention flow from the output token back to the input grid.

One method treats the model as a black box; the other reads the internals of the very block we propose. Both localise disease regions and ignore the background (Figures 3-4): the Occlusion map lights up on diseased tissue - rot lesions for anthracnose, spot clusters for leaf spot, chlorotic areas for yellow leaf - while Attention Rollout concentrates on the leaf/fruit centre. Agreement between two mechanisms sharing no assumptions is evidence that the accuracy reflects genuine disease features rather than an artefact of one technique or a background cue. The cut point of Section 3.2 pays a direct dividend: with a 14x14 grid the Rollout map has **196 cells instead of 49** - fine enough to delineate a symptom region rather than merely indicate a quadrant.

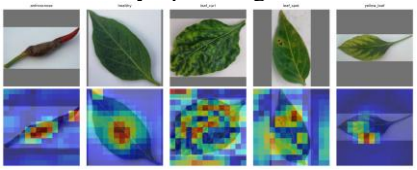


Fig. 3. Occlusion Sensitivity for the five disease classes (top: input; bottom: sensitivity overlay). Bright areas are regions whose occlusion costs the model the most confidence; the method treats the model as a black box.

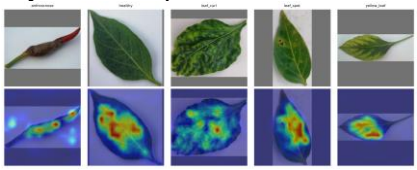


Fig. 4. Attention Rollout of the Tiny Transformer block on the 14x14 grid; attention concentrates on the object centre (leaf/fruit), ignoring the background.

5.2. Comparison with a Vision LLM

As an independent cross-check, a general-purpose multimodal LLM (Gemini 3.1 Flash Lite, accessed 8 July 2026) classifies the raw images zero-shot; this is a feasibility con-

trast, not a peer comparison (the LLM is untrained here, our model specialized). Over the **full 373-image test set** (all calls returned successfully) the LLM reaches **78.55%** against **98.93%**, with errors concentrated on the `leaf_curl`/`yellow_leaf` boundary our model also finds hardest. The specialized model additionally runs offline at near-zero marginal cost with structured five-class outputs, which a cloud LLM cannot offer on-device. Both systems are scored on the **same 373 images**.

On latency. The ~103x figure contrasts a **local 20.59 ms inference** with a **2,117 ms median cloud API round-trip**. These are not like-for-like: the round-trip includes network latency, server-side queueing and provider-side batching, none of which are computation by the LLM. It is a **system-level latency comparison** - what a user of each deployment would actually wait - not a claim that one architecture computes ~103x faster. The accuracy gap is a fair comparison; the latency gap is a deployment-mode comparison. A commercial endpoint is also a moving target: versions update continuously, so this is reproducible only against the stated version and access date.

Table 10. Proposed model versus Gemini 3.1 Flash Lite on the full 373-image test set (accessed 8 July 2026).

System	Accuracy	Latency/image
ChiliEdgeNet (local CPU)	**98.93%**	**20.59 ms** (local inference)
Gemini 3.1 Flash Lite	78.55%	2,117 ms (median API round-trip)

6. Limitations

Statistical resolution of the test set. With 373 images a single image is worth 0.27 pp and the best model errs on only 4. Differences of 0.3-1 pp - which separate several models in Table 3 - correspond to 1-4 images and lie inside the noise band of a single split. The 95% Wilson interval for our augmented run is [97.28, 99.58], and the base architecture [5] falls inside it. This is why the paper does **not** rank models by clean accuracy and does **not** claim an accuracy improvement over [5]; the claim it does make - 35% fewer parameters - is exact arithmetic. We use a fixed single split with no k-fold cross-validation, so our position in Table 3 reads as *among the strongest under a shared protocol*, not a statistically established ordering.

Run count and statistical power. The proposed model and all ablations - and the base architecture [5] they are compared against - come from a single run (seed 42). We compensate with per-image paired McNemar tests and Wilson intervals, but the honest summary is that **no ablation comparison in this paper survives Bonferroni correction**: the strongest are RMSNorm-vs-LayerNorm under blur ($p = 0.019$) and the Transformer block under low light ($p = 0.070$), against corrected thresholds of 0.006 and 0.008. Sections 4.4.1-4.4.2 are therefore design-informing observations, not established effects. Establishing each technique's individual value conclusively requires multiple

seeds and a non-saturated dataset. The parameter-count claims are unaffected - they are exact.

Coverage of the degradation benchmark. This run evaluates two degradations, blur and brightness reduction. Both model geometric and photometric capture failures, but neither models **sensor-level** degradation (ISO noise, JPEG compression), which stresses the high-frequency content a convolutional stack relies on. Robustness conclusions here are scoped to the blur and low-light axes.

Domain gap. ChiliEdge-DS is photographed against a white background (Section 3.1), a controlled acquisition unlike a phone camera in a field. Section 4.3 probes **part** of this gap and the model retains 91.4-99.2% across the degraded conditions versus 98.9% clean. But those transforms simulate **acquisition degradation only**, not **scene-level domain shift**: a blurred white-background image is still a white-background image. Two properties of real field imagery remain **unevaluated**: (i) cluttered leaf-and-soil backgrounds instead of a uniform backdrop, and (ii) unconstrained viewpoints. Our 98.9% clean accuracy should therefore **not** be read as an estimate of in-field deployment accuracy. Collecting even a small natural-background test set to quantify the transfer drop is a necessary next step.

Hardware. The 20.59 ms/image figure is measured on a server-class Kaggle VM CPU (Intel(R) Xeon(R) @ 2.00 GHz, 4 logical cores), **not** edge hardware. A phone or single-board computer has far fewer cores, a lower clock, and a thermal budget that throttles sustained inference, so on-device latency would be substantially higher. The paper establishes the model is *small and cheap enough* to be a realistic edge candidate (2.7 MB, no GPU required); it does not establish measured on-device performance.

7. Conclusion

This paper proposed ChiliEdgeNet, a **671,432-parameter** CNN-Transformer hybrid for chili pepper disease recognition running fully offline on a CPU (2.7 MB, 20.59 ms/image), reaching 98.39% baseline and 98.93% augmented - higher than the base architecture [5] (98.12% at the same seed and protocol) by **+0.81 pp**, while using **35% fewer parameters**. The core contribution is achieving both at once: higher accuracy and a lighter model; the parameter figure alone is exact and needs no test, while the 3-image accuracy gap should be read within the test set's noise band (Section 6).

The methodological contribution is to audit the design component by component instead of reporting the assembled model, using retrain-from-scratch ablation and per-image paired testing on clean images plus two degraded capture conditions. Four findings emerge, three against our own initial expectations:

1. **Two of the three LLM techniques are free; one is not.** 2D-RoPE matches a learned positional table and GQA matches multi-head attention on every axis, together saving **41,472 parameters** at no measurable cost. RMSNorm gives up **2.95 pp under blur** to LayerNorm ($p = 0.019$) to save 384 parameters - the weakest of the three, and one a revision should reconsider.

2. **The Transformer block contributes robustness, not accuracy - and only marginally.** Retraining without it costs 1.61 pp under low light ($p = 0.070$, a trend) and 0.27 pp - one image - on clean data ($p = 1.000$).
3. **Not every inherited block helps.** The Triplet Attention block from [5] harmed accuracy under blur and low light with statistical significance - a negative result we acted on, removing it from the design rather than reporting around it.
4. **On this dataset, low-light robustness comes from the training recipe, not the architecture.** Augmentation moves low-light accuracy from 89.81% to 99.20% (+9.38 pp, 35 images) - an order of magnitude more than any architectural component measured here.

The backbone cut point remains the most consequential architectural decision: `features[1:14]` preserves a 14x14 grid (196 tokens) instead of letting the stride-32 stage collapse it to 7x7, and yields a four-times finer attention map. Future work: extending the degradation benchmark to sensor-level noise and JPEG compression, a natural-background test set to quantify the domain gap, confirmation across seeds and on harder non-saturated datasets, and latency/memory measurement on physical edge hardware after quantization.

References

[1] R. Maurya, S. Mahapatra, and L. Rajput, "A lightweight meta-ensemble approach for plant disease detection suitable for IoT-based environments," *IEEE Access*, vol. 12, pp. 28096-28108, 2024. DOI: 10.1109/ACCESS.2024.3367443.

[2] M. M. Rahman et al., "MaizeFormerX: A lightweight vision transformer with cross-scale attention for explainable maize leaf disease diagnosis," *Scientific Reports*, 2026. DOI: 10.1038/s41598-026-44550-0.

[3] P. Gole, P. Bedi, S. Marwaha, M. A. Haque, and C. K. Deb, "TrIncNet: A lightweight vision transformer network for identification of plant diseases," *Frontiers in Plant Science*, vol. 14, art. no. 1221557, 2023. DOI: 10.3389/fpls.2023.1221557.

[4] D. S. Joseph, P. M. Pawar, and K. Chakradeo, "Real-time plant disease dataset development and detection of plant disease using deep learning," *IEEE Access*, vol. 12, pp. 16310-16333, 2024. DOI: 10.1109/ACCESS.2024.3358333.

[5] S. Duhan, P. Gulia, N. S. Gill, and E. Narwal, "RTR_Lite_MobileNetV2: A lightweight and efficient model for plant disease detection and classification," *Current Plant Biology*, vol. 42, art. no. 100459, 2025. DOI: 10.1016/j.cpb.2025.100459.

[6] G. Li, Y. Wang, Q. Zhao, P. Yuan, and B. Chang, "PMVT: A lightweight vision transformer for plant disease identification on mobile devices," *Frontiers in Plant Science*, vol. 14, art. no. 1256773, 2023. DOI: 10.3389/fpls.2023.1256773.

[7] S. Bhagat, M. Kokare, V. Haswani, P. Hambarde, T. Taori, P. H. Ghante, and D. K. Patil, "Advancing real-time plant disease detection: A lightweight deep learning approach and novel dataset for pigeon pea crop," *Smart Agricultural Technology*, vol. 7, art. no. 100408, 2024. DOI: 10.1016/j.atech.2024.100408.

[8] B. Wang, C. Zhang, Y. Li, C. Cao, D. Huang, and Y. Gong, "An ultra-lightweight efficient network for image-based plant disease and pest infection detection," *Precision Agriculture*, vol. 24, pp. 1836-1861, 2023. DOI: 10.1007/s11119-023-10020-0.

[9] B. Yang et al., "A novel plant type, leaf disease and severity identification framework using CNN and transformer with multi-label method," *Scientific Reports*, vol. 14, art. no. 11664, 2024. DOI: 10.1038/s41598-024-62452-x.
[10] S. Hemalatha and J. J. B. Jayachandran, "A multitask learning-based vision transformer for plant disease localization and classification," *International Journal of Computational Intelligence Systems*, vol. 17, art. no. 188, 2024. DOI: 10.1007/s44196-024-00597-3.
[11] U. Mishra, A. Pandey, Logeswari G, and Tamilarasi K, "Deep learning-based disease detection in potato and mango leaves: A comparative study of CNN, AlexNet, ResNet, and EfficientNet," *Scientific Reports*, 2025. DOI: 10.1038/s41598-025-32607-5.
[12] J. Rahaman, P. Paul, A. Chowdhury, and M. Quamruzzaman, "A customized MobileNetV3Large-based deep learning framework for plant disease detection," *Discover Artificial Intelligence*, vol. 6, art. no. 110, 2026. DOI: 10.1007/s44163-025-00733-8.
[13] Karthik, R. Menaka, M. S. Kathiresan, and M. Anirudh, "GrapeLeafNet: A dual-track feature fusion network with Inception-ResNet and Shuffle-Transformer for accurate grape leaf disease identification," *IEEE Access*, vol. 12, pp. 19612-19624, 2024. DOI: 10.1109/ACCESS.2024.3361044.
[14] R. Sujatha, S. Krishnan, J. M. Chatterjee, and A. H. Gandomi, "Advancing plant leaf disease detection integrating machine learning and deep learning," *Scientific Reports*, vol. 15, art. no. 11552, 2025. DOI: 10.1038/s41598-024-72197-2.
[15] M. Ali, S. Mouatassim, M. El Haji, and J. Benhra, "Plant disease detection using vision transformers," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 15, no. 2, pp. 2334-2344, 2025. DOI: 10.11591/ijece.v15i2.pp2334-2344.
[16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510-4520. DOI: 10.1109/CVPR.2018.00474.
[17] D. Misra, T. Nalamada, A. U. Arasanipalai, and Q. Hou, "Rotate to attend: Convolutional triplet attention module," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, 2021, pp. 3139-3148. DOI: 10.1109/WACV48630.2021.00318.
[18] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, art. no. 127063, 2024. DOI: 10.1016/j.neucom.2023.127063.
[19] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 4895-4901. DOI: 10.18653/v1/2023.emnlp-main.298.
[20] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019. [Online]. Available: https://arxiv.org/abs/1910.07467.
[21] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618-626. DOI: 10.1109/ICCV.2017.74.
[22] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020, pp. 4190-4197. DOI: 10.18653/v1/2020.acl-main.385.
[23] "ChiliEdge-DS: Chili Pepper Leaf Disease Dataset," Kaggle, 2026. [Online]. Available: https://www.kaggle.com/datasets/tminhhi/ot-21-06.
[24] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 818-833. DOI: 10.1007/978-3-319-10590-1_53.